

BaoHoneyPotar – Handbuch

Stand: 10.02.2026

BaoHoneyPotar ist ein System-Plugin für Joomla, das Formular-Spam per Honeypot-Feld und Token-Prüfung reduziert. Es blockiert verdächtige POST-Requests serverseitig mit **HTTP 403** und einer JSON-Antwort.

Produkt: Joomla System-Plugin „BaoHoneyPotar“ (Honeypot-Spamschutz)

Autor: jumbo125

Copyright: © 2025 jumbo125

Lizenz: GNU GPL v2 oder höher

Quickguide

1. **Installieren:** Plugin als Joomla-Erweiterung installieren.
2. **Aktivieren:** Plugin in *Erweiterungen* → *Plugins* aktivieren.
3. **Secret setzen:** In den Plugin-Optionen ein **Secret** hinterlegen (Pflicht).
4. **Auto-Insert (optional):** `auto_insert = 1` aktivieren, um das JS automatisch zu laden.
5. **Testen:**
 - Formular normal absenden (soll funktionieren).
 - Honeypot-Feld befüllen oder Token manipulieren (muss blockieren).
6. **Debug (optional):** `debug = 1` aktivieren und Logdatei `honeypot-debug.txt` im Joomla-Root prüfen.

Tipp: Wenn du das Dokument als PDF erzeugst, nutze im Browser „Drucken → Als PDF speichern“ oder deinen Online-Dienst. Das Layout



1. Zweck und Funktionsprinzip

Das Plugin kombiniert drei Bausteine:

- **Dynamisches Honeypot-Feld** (Name beginnt mit einem Präfix, z.B. `hp_...`)
- **Token** (SHA-256 über Feldname + Secret)
- **Zeitprüfung** (Block bei extrem schnellen Submits, Standard: < 3 Sekunden)

Bots füllen oft alle Felder oder umgehen JS-Logik. Durch ein unsichtbares Feld plus Token und Mindestzeit werden viele automatisierte Submits erkannt.

2. Voraussetzungen

- Joomla (System-Plugin)
- PHP 8+ (wegen `str_contains`, `str_starts_with`, `random_bytes`)
- Schreibrechte im Joomla-Root, falls Debug-Log aktiviert wird (`honeypot-debug.txt`)

3. Installation

1. ZIP-Paket im Joomla-Backend hochladen: *System* → *Installieren* → *Erweiterungen*.
2. Plugin aktivieren: *Erweiterungen* → *Plugins* → *System - BaoHoneyPotar*.
3. Parameter konfigurieren (mindestens `secret`).

4. Konfiguration

4.1 Pflichtparameter

secret

Wird zur Token-Berechnung verwendet. Ohne Secret findet keine verlässliche Prüfung statt.

4.2 Optionale Parameter

secret_praefix (Default: `hp`)

Präfix für Feldnamen. Intern wird ein Unterstrich ergänzt: `hp_`.

auto_insert (0/1)

Wenn aktiv, wird das JS über den WebAssetManager automatisch eingebunden.

debug (0/1)

Schreibt detaillierte Einträge nach `JPATH_SITE/honeypot-debug.txt`.

honeypot_exceptions

Kommagetrennte Liste von URI-Teilstrings. Wenn `REQUEST_URI` einen Eintrag enthält, wird die Prüfung übersprungen.

Beispiel: `com_users, /api/`

honeypot_skip_keys

Kommagetrennte POST-Keys. Wenn einer davon im POST vorkommt, wird die Prüfung übersprungen.

Beispiel: `g-recaptcha-response, csrf_token`

5. Ablauf der Server-Prüfung (onAfterInitialise)

Die Prüfung läuft nur unter diesen Bedingungen:

- Frontend (`$app->isClient('site')`)
- Request-Methode ist `POST`

5.1 Exceptions (URI)

Wenn die aktuelle `REQUEST_URI` einen Eintrag aus `honeypot_exceptions` enthält, wird nicht geprüft.

5.2 Skip-Keys (POST)

Wenn einer der POST-Keys aus `honeypot_skip_keys` vorhanden ist, wird nicht geprüft.

5.3 Honeypot-Feld finden

Es wird das erste POST-Feld gesucht, dessen Name:

- mit dem Präfix beginnt (z.B. `hp_`)
- nicht `hp_token` und nicht `hp_token_time` ist

Dieses Feld ist das „Honeypot“-Feld.

5.4 Token- und Zeitfelder

Ausgehend vom gefundenen Feldname (Beispiel: `hp_ab12cd34ef`) werden weitere Keys gelesen:

- Honeypot-Wert: `hp_ab12cd34ef`
- Token: `hp_ab12cd34eftoken`
- Timestamp: `hp_ab12cd34ef_token_time`

Erwarteter Token:

```
expected = sha256(honeypotField + secret)
```

5.5 Block-Regeln

Ein Request wird mit **HTTP 403** und JSON blockiert, wenn:

- Honeypot-Feld nicht leer ist
- Token nicht dem erwarteten Wert entspricht
- Timestamp vorhanden ist und $(\text{now} - \text{timestamp}) < 3$ Sekunden

Block-Antwort (vereinfacht):

```
{
  "success": false,
  "message": "..."
}
```

6. Automatisches Einbinden des JavaScripts (onBeforeCompileHead)

Wenn `auto_insert = 1`:

- WebAsset Registry wird aus `media/plg_system_baohoneypotar/joomla.asset.json` geladen
- Script `honeypot-loader` wird aktiviert
- Option `baohoneypotar.debug` wird per `addScriptOptions` an JS übergeben

Hinweis: Das eigentliche Verhalten hängt von deinem JS (`honeypot-loader`) ab, das die Felder anlegt und die Werte befüllt.

7. AJAX-Endpunkt zur Feldgenerierung (onAjaxBaohoneypotar)

Der AJAX-Handler erzeugt serverseitig:

- Feldname: `praefix + randomHex(10)` (z.B. `hp_a1b2c3d4e5`)
- Token: `sha256(field + secret)`

Antwort:

```
{
  "field": "hp_a1b2c3d4e5",
  "token": "<sha256>"
}
```

Wenn `secret` fehlt, kommt eine Fehlerantwort (HTTP 400).

8. Debugging und Logging

Wenn `debug = 1` , wird nach `honeypot-debug.txt` geschrieben, u.a.:

- gefundener Feldname
- Honeypot-Wert
- Token / erwarteter Token
- Zeitdifferenz
- URI und Request-Methode
- bei Block: User-Agent und POST-Keys

Tipp: Logdatei nach Tests wieder leeren, um die Auswertung zu erleichtern.

9. Sicherheitshinweise

- Secret nie öffentlich machen (nicht im Repo, nicht im JS).
- Debug-Log kann sensible Daten enthalten (POST-Werte). Debug nur kurzzeitig aktivieren.
- Honeypot ist kein vollständiger Ersatz für Captcha oder Rate Limiting, aber reduziert Bot-Traffic deutlich.

10. Troubleshooting

Formular wird immer blockiert

- Secret gesetzt?
- JS fügt Token und Timestamp korrekt hinzu?
- Wird das richtige Feldname-Schema verwendet (`<field>token` und `<field>_token_time`)?
- Wird das Formular extrem schnell abgesendet (Auto-Fill/Auto-Submit)?

Keine Wirkung

- Ist das Plugin aktiviert?
- Passiert der Request im Frontend?
- Ist es wirklich ein POST?
- Greift eine Exception/Skip-Key Regel?

Debug-Log erscheint nicht

- Schreibrechte im Joomla-Root?
- Parameter `debug = 1` aktiv?

11. Dateien und Architektur

Typische Bausteine im Projekt:

- **Plugin-Klasse:** `namespace Joomla\Plugin\System\Baohoneypotar\Plugin`
 - `onAfterInitialise()` Prüfung/Block
 - `onBeforeCompileHead()` WebAssets/JS
 - `onAjaxBaohoneypotar()` Feld/Token Ausgabe
- **Service Provider** (Dependency Injection)
 - registriert `PluginInterface` und baut Plugin-Instanz mit Dispatcher und Config
- **Assets**
 - `media/plg_system_baohoneypotar/joomla.asset.json`
 - JS `honeypot-loader` (nicht im Codeausschnitt enthalten)

Ende

[↑ nach oben](#)